

Lecture-#8

Natural Language Processing

A8706

Syntax Analysis

Treebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

- Parsing recovers information that is not explicit in the input sentence.
- This implies that a parser requires some **knowledge(syntactic rules)** in addition to the input sentence about the kind of syntactic analysis that should be produced as output.
- **One method to provide such knowledge** to the parser is to write down a grammar of the language – **a set of rules** of syntactic analysis **as a CFGs**.
- In natural language, it is far **too complex** to simply **list all the syntactic rules** in terms of a CFG.
- The second knowledge acquisition problem- **not only do we need to know the syntactic rules** for a particular language, but we also need to know which **analysis is the most plausible**(probably) for a given input sentence.
- The **construction of treebank is a data driven approach to syntax analysis** that allows us to address both of these knowledge acquisition bottlenecks in one stroke.
- A treebank is simply a **collection of sentences** (also called a corpus of text), where each sentence is provided a **complete syntax analysis**.
- The syntactic analysis for each sentence has been judged by a **human expert** as the most possible analysis for that sentence.

Treebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

- ❖ A lot of care is taken during the **human annotation process** to ensure that a consistent treatment is provided across the treebank for related grammatical phenomena.
- ❖ There is no **set of syntactic rules** or **linguistic grammar** explicitly provided by a treebank, and typically there is no list of syntactic constructions provided explicitly in a treebank.
- ❖ **A detailed set of assumptions about the syntax is typically used** as an **annotation** guideline to help the human experts produce the **single-most plausible syntactic analysis** for each sentence in the corpus.
- ❖ Treebanks provide a **solution to the two kinds** of knowledge acquisition **bottlenecks**.
- ❖ Treebanks solve the first knowledge acquisition problem of **finding the grammar** underlying the syntax analysis because the **syntactic analysis is directly** given instead of a grammar.
- ❖ In fact, the parser does not necessarily need any explicit grammar rules as long as it can faithfully produce a syntax analysis for an input sentence.
- ❖ Treebank solve the second knowledge acquisition problem as well.
- ❖ Because each sentence in a treebank has been given its **most plausible(probable)** syntactic analysis, **supervised machine learning methods** can be used **to learn a scoring function** over **all possible** syntax analyses.

Trebanks: A Data-Driven Approach to Syntax,



Learning from Experience, Sharing with Passion

CFG: CFG consists of a finite set of grammar rules having the following four components

- ❖ Set of Non-Terminals
- ❖ Set of Terminals
- ❖ Set of Productions
- ❖ Start Symbol

Set of Non-terminals

It is represented by V . The non-terminals are syntactic variables that denote the sets of strings, which helps in defining the language that is generated with the help of grammar.

Set of Terminals

It is also known as tokens and represented by Σ . Strings are formed with the help of the basic symbols of terminals.

Set of Productions

It is represented by P . The set gives an idea about how the terminals and nonterminals can be combined. Every production consists of the following components:

Non-terminals,

Arrow,

Terminals (the sequence of terminals).

The left side of production is called non-terminals while the right side of production is called terminals.

Start Symbol

The production begins from the start symbol. It is represented by symbol S . Non-terminal symbols are always designated as start symbols.

Treebanks: A Data-Driven Approach to Syntax,

Left Most Derivation (LMD) and Derivation Tree :

- ❖ Leftmost derivation of a string from starting symbol S is done by replacing leftmost non-terminal symbol by RHS of corresponding production rule.
- ❖ For example, the leftmost derivation of string $abab$ from grammar G above is done as : $\underline{S} \Rightarrow a\underline{S}b \Rightarrow ab\underline{S}ab \Rightarrow abab$
- ❖ The symbols underlined are replaced using production rules.

Right Most Derivation (RMD) :

- ❖ Rightmost derivation of a string from starting symbol S is done by replacing rightmost non-terminal symbol by RHS of corresponding production rule.
 - ❖ e.g.; The rightmost derivation of string $abab$ from grammar G above is done as : $\underline{S} \Rightarrow S\underline{S} \Rightarrow Sa\underline{S}b \Rightarrow \underline{S}ab \Rightarrow a\underline{S}bab \Rightarrow abab$
 - ❖ The symbols underlined are replaced using production rules.
- The derivation tree for $abab$ using rightmost derivation

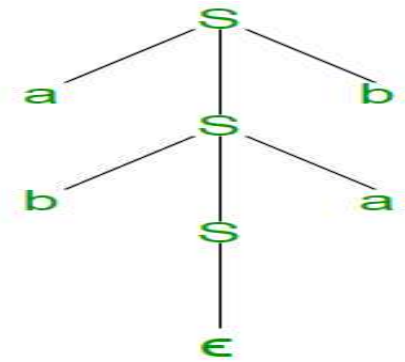


Figure 1

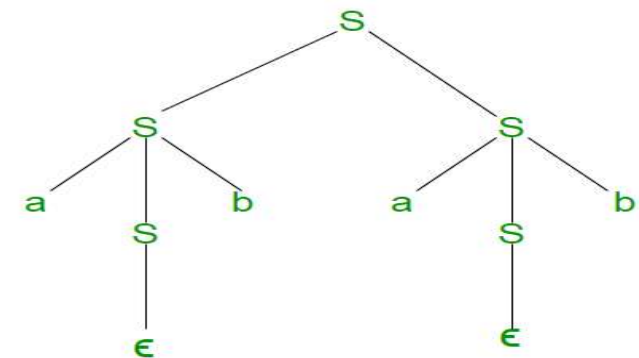


Figure 2

Ambiguous Context Free Grammar :

- ❖ A context free grammar is called ambiguous if there exists more than one LMD or more than one RMD for a string which is generated by grammar.
- ❖ There will also be more than one derivation tree for a string in ambiguous grammar.
- ❖ The grammar described above is ambiguous because there are two derivation trees (Figure 1 and Figure 2).
- ❖ There can be more than one RMD for string abab which are:
$$\underline{S} \Rightarrow S\underline{S} \Rightarrow Sa\underline{S}b \Rightarrow \underline{S}ab \Rightarrow a\underline{S}bab \Rightarrow abab$$
$$\underline{S} \Rightarrow a\underline{S}b \Rightarrow ab\underline{S}ab \Rightarrow abab$$

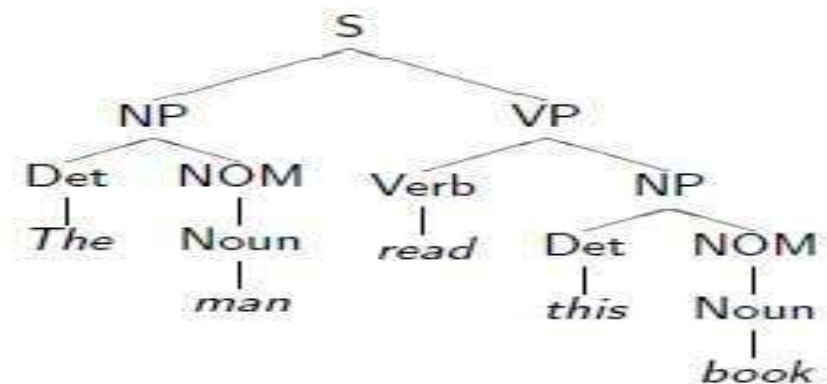
Treebanks: A Data-Driven Approach to Syntax,

Application of grammar rewrite rules

$S \rightarrow NP VP$	$Det \rightarrow that \mid this \mid a \mid the$
$S \rightarrow Aux NP VP$	$Noun \rightarrow book \mid flight \mid meal \mid man$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid read$
$NP \rightarrow Det NOM$	$Aux \rightarrow does$
$NOM \rightarrow Noun$	
$NOM \rightarrow Noun NOM$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb NP$	

$S \rightarrow NP VP$
 $\rightarrow Det NOM VP$
 $\rightarrow The NOM VP$
 $\rightarrow The Noun VP$
 $\rightarrow The man VP$
 $\rightarrow The man Verb NP$
 $\rightarrow The man read NP$
 $\rightarrow The man read Det NOM$
 $\rightarrow The man read this NOM$
 $\rightarrow The man read this Noun$
 $\rightarrow The man read this book$

Parse tree



Text Normalization

Removing Stopwords :

Stop Words: A stop word is a commonly used word that a search engine has been programmed to ignore, both when indexing entries for searching and when retrieving them as the result of a search query.

```
import nltk
from nltk.corpus import stopwords
print(stopwords.words('english'))
```


Text Normalization

Removing Stopwords :

['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above', 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn', "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]

Total-179

Text Normalization

Removing Stopwords :

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
example_sent = """This is a sample sentence, showing off the stop words filtration."""
stop_words = set(stopwords.words('english'))
word_tokens = word_tokenize(example_sent)
# converts the words in word_tokens to lower case and then checks whether
#they are present in stop_words or not
filtered_sentence = [w for w in word_tokens if not w.lower() in stop_words]
#with no lower case conversion
filtered_sentence = []
for w in word_tokens:
    if w not in stop_words:
        filtered_sentence.append(w)
print(word_tokens)
print(filtered_sentence)
```

Text Normalization

Removing Stopwords:

['This', 'is', 'a', 'sample', 'sentence', ',', 'showing', 'off', 'the', 'stop', 'words', 'filtration', '.']

['This', 'sample', 'sentence', ',', 'showing', 'stop', 'words', 'filtration', '.']

Text Normalization

Correcting Words:

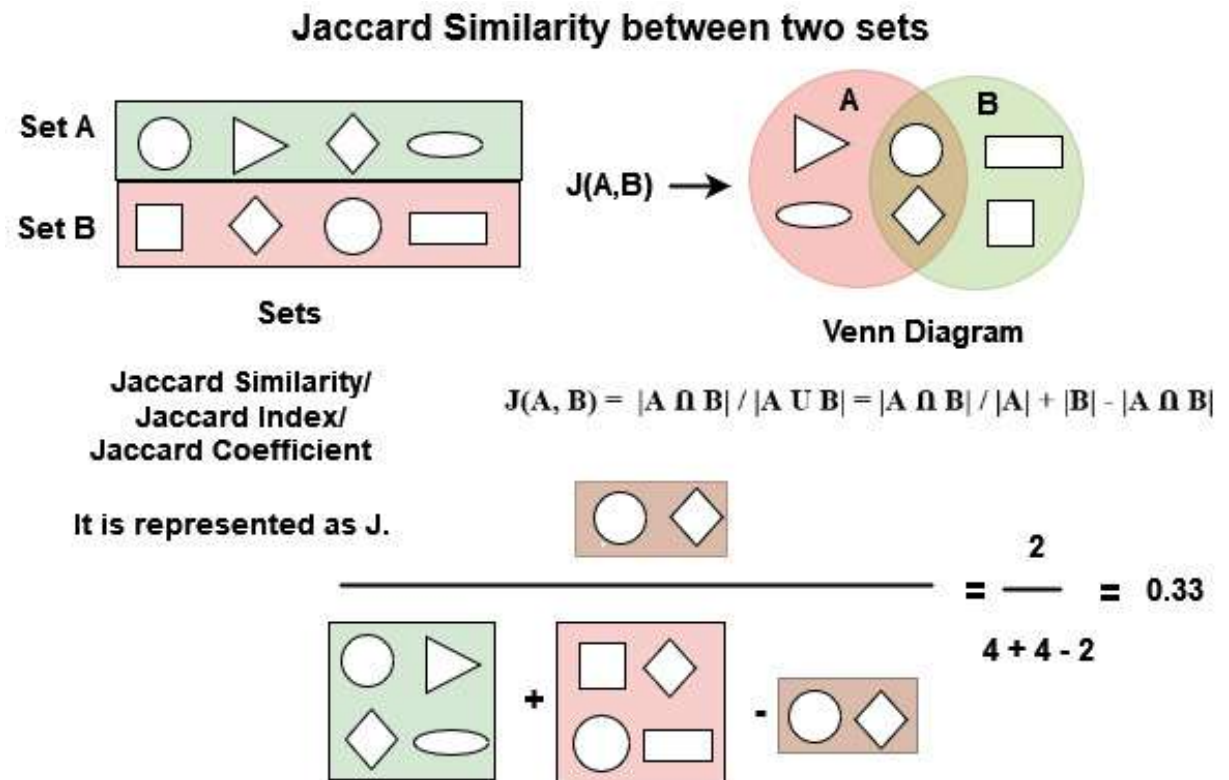
- ❖ Each method takes a list of misspelled words and gives the suggestion of the correct word for each incorrect word.
- ❖ It tries to find a word in the list of correct spellings that has the shortest distance and the same initial letter as the misspelled word.
- ❖ It then returns the word which matches the given criteria.
- ❖ The methods can be differentiated on the basis of the distance measure they use to find the closest word. 'words' package from nltk is used as the dictionary of correct words.
- ❖ **Using Jaccard distance Method:**
- ❖ It is mainly used to measure similarities between two sets or between two asymmetric binary vectors.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

- ❖ **Jaccard distance=1- J(A, B)** where J(A, B) is Jaccard index

Text Normalization

Correcting Words:



Jaccard distance = $1 - J(A, B) = 1 - 0.33 = 0.66$

Text Normalization

Correcting Words:

```
# importing the nltk suite
import nltk
# importing jaccard distance
# and ngrams from nltk.util
from nltk.metrics.distance import jaccard_distance
from nltk.util import ngrams
# Downloading and importing
# package 'words' from nltk corpus
nltk.download('words')
from nltk.corpus import words
correct_words = words.words()
w1 = set('cat')
w2 = set('cap')
nltk.jaccard_distance(w1, w2)
```

O/P: 0.5

Text Normalization

Correcting Words:

```
# list of incorrect spellings
# that need to be corrected
incorrect_words=['happy', 'azmaing', 'intelliengt']

# loop for finding correct spellings
# based on jaccard distance
# and printing the correct word
for word in incorrect_words:
    temp = [(jaccard_distance(set(ngrams(word, 2)),
                                set(ngrams(w, 2))),w)
             for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

O/P:

happy

Amazing

intelligent

Text Normalization

`ngrams(word, 2):`

- This generates **bigrams** (sequences of two consecutive characters) from the given word. For example, for the word "happy", it will generate the bigrams ['ha', 'ap', 'pp', 'py'].

`set(ngrams(word, 2)):`

- The `ngrams()` function produces a list, but `set()` converts that list into a set of unique bigrams. This is useful because the Jaccard distance works on set comparisons, not lists. For example, `set(['ha', 'ap', 'pp', 'py'])` for the word "happy".

Text Normalization

•Example: Jaccard Distance Between 'MAKE' and 'CAKE'

- **Step 1: Convert words to sets of bigrams (2-grams):**
 - First, convert the words into sets of character bigrams (two consecutive characters).
- For 'MAKE':
 - Bigrams of MAKE={ 'MA', 'AK', 'KE' }
- For 'CAKE':
 - Bigrams of CAKE={ 'CA', 'AK', 'KE' }
- **Step 2: Find the intersection:**
 - The **intersection** is the set of common bigrams between the two sets.
 - $A \cap B = \{ 'AK', 'KE' \}$
- So, the size of the intersection is 2.

Text Normalization

•Step 3: Find the union:

- The **union** is the set of all unique bigrams from both sets.

$$- A \cup B = \{'MA', 'AK', 'KE', 'CA'\}$$

– So, the size of the union is 4.

– Step 4: Compute the Jaccard similarity:

$$\text{Jaccard Similarity} = \frac{|A \cap B|}{|A \cup B|} = \frac{2}{4} = 0.5$$

– Step 5: Compute the Jaccard distance:

$$\text{Jaccard Distance} = 1 - \text{Jaccard Similarity} = 1 - 0.5 = 0.5$$

Text Normalization

Correcting Words:

Using Edit distance Method: The distance between the source string and the target string is the minimum number of edit operations (deletions, insertions, or substitutions) required to transform the source into the target. The lower the distance, the more similar the two strings.

importing the nltk suite

import nltk

importing edit distance

from nltk.metrics.distance import edit_distance

Downloading and importing package 'words'

nltk.download('words')

from nltk.corpus import words

correct_words = words.words()

Text Normalization

Correcting Words:

```
import nltk  
w1 = 'cat'  
w2 = 'cap'  
nltk.edit_distance(w1, w2)
```

O/P:1

Input: *str1 = "cat", str2 = "cut"*

Output: *1*

Explanation: *We can convert str1 into str2 by replacing 'a' with 'u'.*

Input: *str1 = "sunday", str2 = "saturday"*

Output: *3*

Explanation: *Last three and first characters are same. We basically need to convert "un" to "atur". This can be done using below three ope*

Text Normalization

Correcting Words:

```
# list of incorrect spellings
# that need to be corrected
incorrect_words=['happpy', 'azmaing', 'intelliengt']
# loop for finding correct spellings
# based on edit distance and
# printing the correct words
for word in incorrect_words:
    temp = [(edit_distance(word, w),w) for w in correct_words if w[0]==word[0]]
    print(sorted(temp, key = lambda val:val[0])[0][1])
```

O/P:

happy

aiming

intelligent

Text Normalization

The **edit distance** (or Levenshtein distance) method is useful in various Natural Language Processing (NLP) and text processing tasks. Here's how it can be applied and why it is important:

1. Spelling Correction:

- **Problem:** Given a misspelled word, how do you suggest the most likely correct word?
- **How Edit Distance Helps:** It calculates the number of changes (insertions, deletions, substitutions) needed to turn the misspelled word into a correct one. The word with the smallest edit distance is likely to be the intended correct word.
- **Example:**
 - Misspelled word: "happpy"
 - Closest correct word by edit distance: "happy" (edit distance of 1)
 - This tells us only one character needs to be removed, making it likely that "happy" was the intended word.

2. Autocorrect Systems:

- **Problem:** When typing on a keyboard or mobile device, people make frequent typos.
- **How Edit Distance Helps:** Autocorrect algorithms use edit distance to suggest the correct word based on minimal changes to the typed word.
- **Example:** Typing "teh" suggests "the" because it has an edit distance of 1 (a simple character swap).

Text Normalization

3. Plagiarism Detection:

- **Problem:** Detecting if two documents are similar even if minor changes (word swaps, misspellings, etc.) are made.
- **How Edit Distance Helps:** By calculating the edit distance between sentences, paragraphs, or words in documents, systems can estimate the level of similarity. A smaller edit distance implies higher similarity.

4. DNA Sequence Analysis:

- **Problem:** Comparing DNA sequences to find similarities or mutations.
- **How Edit Distance Helps:** In bioinformatics, edit distance can be used to quantify the similarity between DNA sequences, helping identify genetic mutations.
- **Example:** Comparing two DNA sequences: "AGCTT" and "ACGTT" would have an edit distance of 1 (a single substitution).

Text Normalization

5. Search Engines & Fuzzy Matching:

- **Problem:** When searching online, users may enter queries with typos or slightly incorrect words.
- **How Edit Distance Helps:** Search engines and systems use edit distance to find results that match closely even when the query has minor errors.
- **Example:** Searching for "recepie" can yield results for "recipe" due to a small edit distance.

6. Text Similarity and Clustering:

- **Problem:** Finding how similar two strings, sentences, or documents are.
- **How Edit Distance Helps:** If the edit distance between two pieces of text is small, they are likely closely related. This is useful in clustering similar text documents or finding paraphrases.
- **Example:** Comparing "Hello World" and "Hello Wrld" gives an edit distance of 1, showing they are very similar.

Text Normalization

7. Named Entity Recognition (NER):

- **Problem:** Identifying proper nouns (like names of people, places) even when there are small spelling differences.
- **How Edit Distance Helps:** In systems that try to recognize names (or brands), slight variations (like missing or extra letters) can still be detected as the same entity if the edit distance is low.

Parts of Speech (POS) Tagging

❖ part-of-speech tagging (POS tagging or PoS tagging or POST), also called grammatical tagging or word-category disambiguation.

❖ Input: I am going.

❖ Output:

with stop word

[('I', 'PRP'), ('am', 'VBP'), ('going', 'VBG'), ('.', '.')]]

Without stop word

[('I', 'PRP'), ('going', 'VBG'), ('.', '.')]]

Where

PRP- Pronoun Personal

VBP-Verb non-3rd personal singular present

VBG-Verb Gerund/ Present Participle

Parts of Speech (POS) Tagging

Sl No.	TAG	DESCRIPTION	EXAMPLE(S)
1	CC	Coordinating Conjunction	<i>and, or</i>
2	CD	Cardinal Number	<i>five, one, 2</i>
3	DT	Determiner	<i>a, the</i>
4	EX	Existential <i>there</i>	<i>there were two cars</i>
5	FW	Foreign Word	<i>d'hoevre, mais</i>
6	IN	Preposition/ Subordinating Conjunction	<i>of, in, on, that</i>
7	JJ	Adjective	<i>quick, lazy</i>
8	JJR	Adjective, comparative	<i>quicker, lazier</i>

Parts of Speech (POS) Tagging

9	JJS	Adjective, superlative	<i>quickest, laziest</i>
10	LS	List item marker	<i>2)</i>
11	MD	Verb, modal	<i>could, should</i>
12	NN	Noun, singular or mass	<i>fox, dog</i>
13	NNS	Noun, plural	<i>foxes, dogs</i>
14	NNP	Noun, proper singular	<i>John, Alice</i>
15	NNPS	Noun, proper plural	<i>Vikings, Indians, Germans</i>
16	PDT	Predeterminer	<i>both the cats</i>
17	POS	Possessive ending	<i>boss's</i>
18	PRP	Pronoun, personal	<i>me, you</i>

Parts of Speech (POS) Tagging

19	PRP\$	Pronoun, possessive	<i>our, my, your</i>
20	RB	Adverb	<i>naturally, extremely, hardly</i>
21	RBR	Adverb, comparative	<i>better</i>
22	RBS	Adverb, superlative	<i>best</i>
23	RP	Adverb, particle	<i>about, up</i>
24	SYM	Symbol	<i>%, \$</i>
25	TO	Infinitival to	<i>how to, what to do</i>
26	UH	Interjection	<i>oh, gosh, wow</i>

Parts of Speech (POS) Tagging

SI No.	TAG	DESCRIPTION	EXAMPLE(S)
27	VB	Verb, base form	<i>run, give</i>
28	VBD	Verb, past tense	<i>ran, gave</i>
29	VBG	Verb, gerund/ present participle	<i>running, giving</i>
30	VBN	Verb, past participle	<i>given</i>
31	VBP	Verb, non-3rd person singular present	<i>I think, I take</i>
32	VBZ	Verb, 3rd person singular present	<i>he thinks, he takes</i>
33	WDT	Wh-determiner	<i>which, whatever</i>
34	WP	Wh-pronoun, personal	<i>who, what</i>
35	WP\$	Wh-pronoun, possessive	<i>whose</i>

Parts of Speech (POS) Tagging

36	WRB	Wh-adverb	<i>where, when</i>
37	NP	Noun Phrase	<i>the brown fox</i>
38	PP	Prepositional Phrase	<i>in between, over the dog</i>
39	VP	Verb Phrase	<i>was jumping</i>
40	ADJP	Adjective Phrase	<i>warm and snug</i>
41	ADVP	Adverb Phrase	<i>also</i>
42	SBAR	Subordinating Conjunction	<i>whether or not</i>
43	PRT	Particle	<i>up</i>
44	INTJ	Interjection	<i>hello</i>
45	PNP	Prepositional Noun Phrase	<i>over the dog, as of today</i>
46	-SBJ	Sentence Subject	<i>the fox jumped over the dog</i>
47	-OBJ	Sentence Object	<i>the fox jumped over the dog</i>

Parts of Speech (POS) Tagging

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = " Everything is all about money."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
    # Word tokenizers is used to find the words
    # and punctuation in a string
    wordsList = nltk.word_tokenize(i)
# removing stop words from wordList
wordsList = [w for w in wordsList if not w in stop_words]
# Using a Tagger. Which is part-of-speech
# tagger or POS-tagger.
tagged = nltk.pos_tag(wordsList)
print(tagged)
```

O/P:

```
[('Everything', 'VBG'),
('money', 'NN'),
('.', '.')]

```


Parts of Speech (POS) Tagging

with stop words

```
import nltk
# download required nltk packages
# required for tokenization
nltk.download('punkt')
# required for parts of speech tagging
nltk.download('averaged_perceptron_tagger')

# input text
sentence = """"Today morning, I am Very Happy.""""

# tokene into words
tokens = nltk.word_tokenize(sentence)

# parts of speech tagging
tagged = nltk.pos_tag(tokens)

# print tagged tokens
print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','),
('I', 'PRP'),
('am', 'VBP'),
('Very', 'RB'),
('Happy', 'JJ'), (',', ',')]
```

Parts of Speech (POS) Tagging

```
# Without stop words
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize, sent_tokenize
stop_words = set(stopwords.words('english'))
txt = "Today morning, I am Very Happy."
# sent_tokenize is one of instances of
# PunktSentenceTokenizer from the nltk.tokenize.punkt module
tokenized = sent_tokenize(txt)
for i in tokenized:
# Word tokenizers is used to find the words
# and punctuation in a string
    wordsList = nltk.word_tokenize(i)
    # removing stop words from wordList
    wordsList = [w for w in wordsList if not w in stop_words]
    # Using a Tagger. Which is part-of-speech
    # tagger or POS-tagger.
    tagged = nltk.pos_tag(wordsList)
    print(tagged)
```

O/P:

```
[('Today', 'NN'),
('morning', 'NN'),
(',', ','), ('I', 'PRP'),
('Very', 'RB'),
('Happy', 'JJ'), (':', ':')]
```

